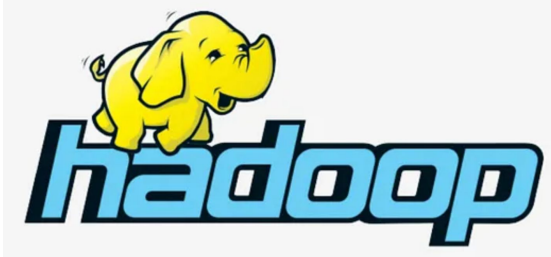


DISTRIBUTED AND CLOUD COMPUTING

LAB 9: HADOOP DFS & MAP-REDUCE



What is Hadoop?

A framework for **distributed storing** and **processing** of **HUGE datasets** in HPC clusters

Framework: a collection of software libraries, tools and technical documents

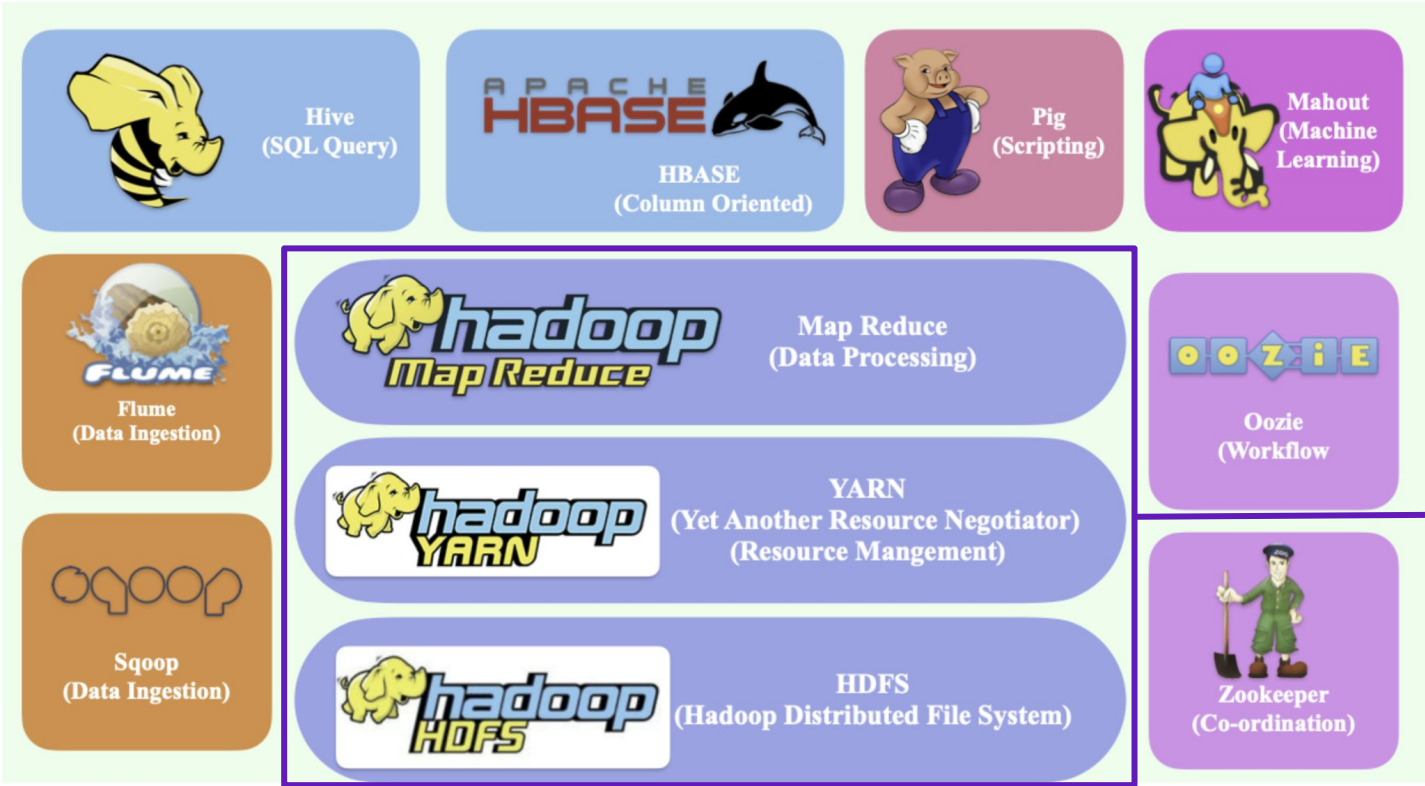
Principles and assumption behind hadoop:

1. Hardware failure is the norm rather than the exception
2. High throughput over low latency of data access
3. Assumes a “write-once-read-many” access model for files
4. Applications that run on HDFS have large data sets (100s of GBs to Terabytes)

5. Moving Computation is Cheaper than Moving Data

Would you rather **upload a 10Kb script** to 10 servers or **download a 1TB dataset** from them?

The Hadoop Ecosystem



**Our
Focus**

A brief explanation...

Hadoop Distributed File System (HDFS)

- Distributed, Fault-tolerant, Append-only file system

YARN

- Resource Manager and Scheduler
- Handles jobs requests from clients
- Assigns them to nodes for execution
- Manages the resources of the cluster

MapReduce

- A data processing paradigm for key-value data (think hashmaps / dictionaries)



Hadoop Distributed File System (HDFS)

“A distributed, fault-tolerant, write-once-read-many file system”

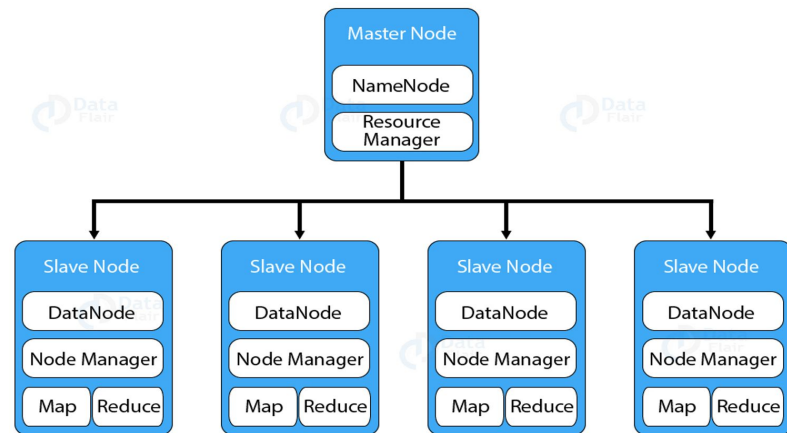
Distributed: data is distributed along many computers

Fault-tolerant: can tolerate **some** node failures without loss of data

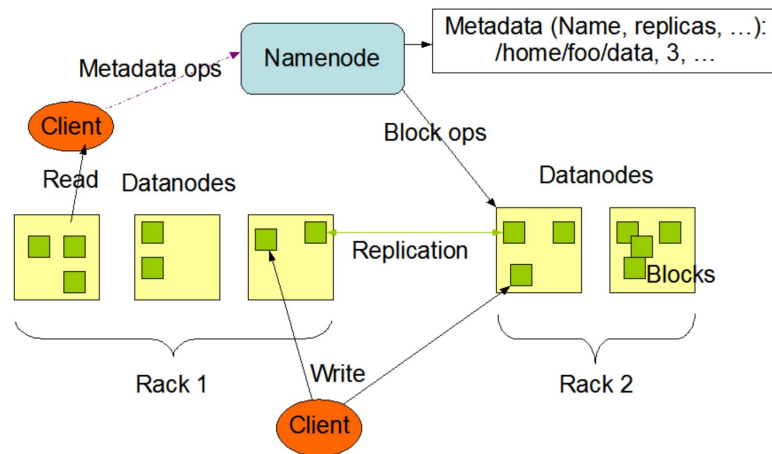
Append-only: does not support modification of existing data

HDFS cluster architecture

- HDFS consists of
 - **1 NameNode** and **MANY DataNodes**
- **NameNode:**
 - Manages how data are distributed among the DataNodes
 - Manages the filesystems Namespace
 - Executes client requests (open, close, rename, etc..)
- **DataNodes:**
 - Store the data
 - Respond to client read and write requests after permission is given from the **NameNode**
- The Namespace of HDFS follows a traditional hierarchical file organisation
 - e.g. /users/name/dataset1



HDFS Architecture



How HDFS store files

- **HDFS** files are split into blocks
 - Default value is 128MB
- These blocks are stored by the DataNodes as **normal files** in their local file system
- Each file has a replication factor (**r**) associated with it
 - The default replication factor is 3

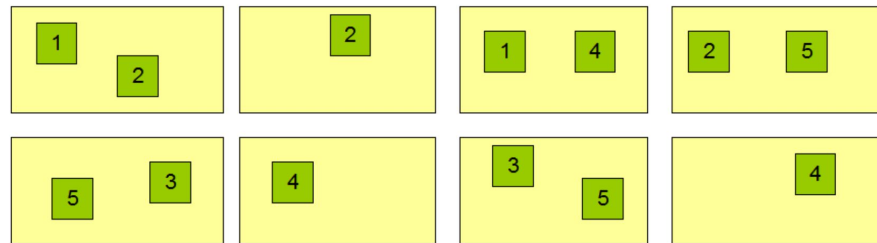
The NameNode ensures that **ALL** blocks of a file are stored in **r** DataNodes

A for a single file we can tolerate **r-1** node failing **at once** before any data is lost!

Block Replication

```
Namenode (Filename, numReplicas, block-ids, ...)  
/users/sameerp/data/part-0, r:2, {1,3}, ...  
/users/sameerp/data/part-1, r:3, {2,4,5}, ...
```

Datanodes



TODO: Set up a hadoop cluster using docker

Use the provided docker-compose file and Makefile to start the compose with the hadoop cluster (**download from Blackboard**)

- Make sure the the parent directory is called 'hadoop'
- Run 'make start' (install make if not installed)
- if the image is not downloading try pulling the image explicitly using

```
$ docker pull apache/hadoop:3.4.1
```

```
WSL: ~ /dncc25/hadoop
$ make start
docker-compose up -d
[*] Running 7/7
✓Network hadoop_default Created 0.1s
✓Container hadoop-datanode-2-1 Started 0.9s
✓Container hadoop-datanode-4-1 Started 0.8s
✓Container hadoop-namenode-1 Started 0.9s
✓Container hadoop-datanode-1-1 Started 0.9s
✓Container hadoop-datanode-3-1 Started 1.0s
✓Container hadoop-resource-manager-1 Started 1.0s
```

Connect to a running node using 'make connect'

- If your parent dir name is different you can modify the Makefile

```
WSL: ~ /dncc25/hadoop
$ make connect
docker exec -it hadoop-datanode-1-1 /bin/bash
bash-4.2$
```

Go to <http://localhost:9870> and checkout the HDFS cluster

Hadoop	Overview	Datanodes	Datanode Volume Failures	Snapshot	Startup Progress	Utilities
Overview 'namenode:8020' (✓active)						
Started:	Wed Nov 12 14:56:11 +0800 2025					
Version:	3.4.1, 14876253054865633060560326794224941					
Compiled:	Wed Oct 09 22:57:00 +0800 2024 by mthaku from branch-3.4.1					
Cluster ID:	GD-e1c2d32-18b-46c3-a0d5-46bdc11d11					
Block Pool ID:	BP-2060316075-172-18-0-5-176293058795					

Summary

From the datanode use the command:
hdfs dfs -<command> <arguments>

To interact with the HDFS

HDFS commands are UNIX-like (ls, cat, mkdir etc..)

All commands can be seen here:

<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/FileSystemShell.html>

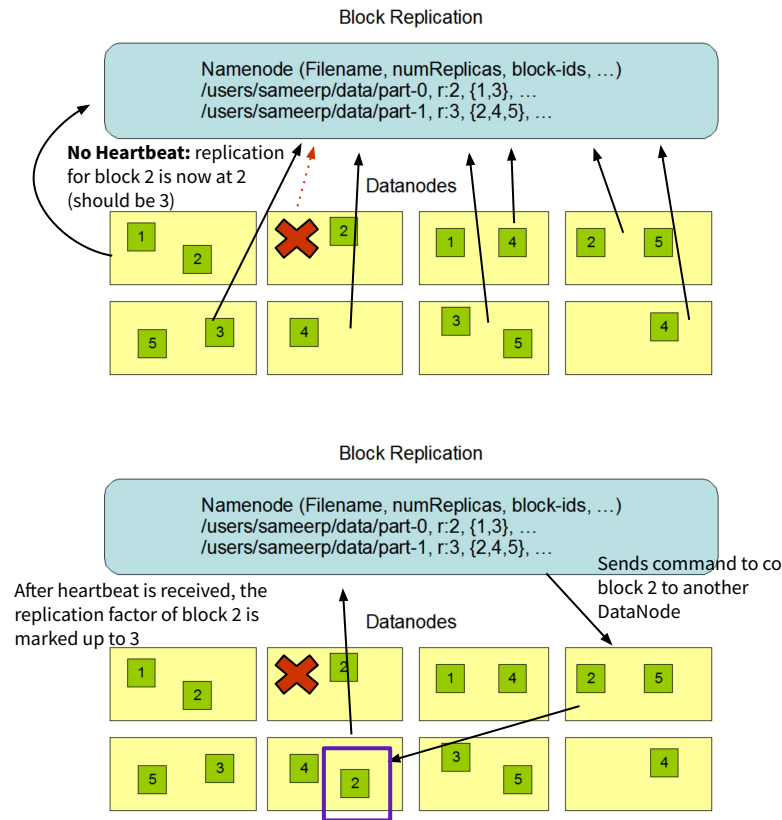
Try creating a local file, uploading it to HDFS and printing its contents.

Once you finish you can read more about HDFS here:

<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

How HDFS achieves fault-tolerance

- DataNodes send heartbeat messages periodically to the NameNode
 - These contain all data blocks stored in that node
- Through heartbeats the NameNode maintains a picture of:
 - where each file (its blocks) is stored
 - The current replication of each block
- If the NameNode stops receiving heartbeats for a DataNode it marks it as failed
- When a node fails the NameNode calculates the replication of all blocks stored in that were stored in that DataNode
 - If any blocks fall under their replication factor the NameNode issues a command to create new copies in new DataNode to achieve replication



MapReduce

Example: Calculating max temperature

- Assume you have to process a weather dataset to find the highest temperature for each year
 - Simple:
 - loop over all entries
 - keep track of highest value for each year
- What if that dataset has data for each hour for over 100 years?
- **We can use MPI and distribute the algorithm!**

But this a problem!

```
raw
|- 1901
|  |- 010010-99999-1901.gz
|  |- 010014-99999-1901.gz
|  |- 010015-99999-1901.gz
|  |- 010016-99999-1901.gz
|  |- 010017-99999-1901.gz
|  |- 010030-99999-1901.gz
|  |- 010040-99999-1901.gz
|  |- 010080-99999-1901.gz
|  |- 010100-99999-1901.gz
|- 1902
|- 1903
|- 1904
|- 1905
|- 1906
|- 1907
|- 1908
|- 1909
|- 1910
.
.
.
|- 2001
```

```
% ls raw/1990 | head
010010-99999-1990.gz
010014-99999-1990.gz
010015-99999-1990.gz
010016-99999-1990.gz
010017-99999-1990.gz
010030-99999-1990.gz
010040-99999-1990.gz
010080-99999-1990.gz
010100-99999-1990.gz
```

```
0067011990999991950051507004...9999999N9+00001+9999999999...
0043011990999991950051512004...9999999N9+00221+9999999999...
0043011990999991950051518004...9999999N9-00111+9999999999...
0043012650999991949032412004...0500001N9+01111+9999999999...
0043012650999991949032418004...0500001N9+00781+9999999999...
```

Example: Calculating max temperature

The root would have to send **many GB's** of data to other processes which is slow and inefficient!

Solution:

- Store everything in HDFS and distribute the computation NOT the data
- MapReduce does exactly that

```
raw
|- 1901
|  |- 010010-99999-1901.gz
|  |- 010014-99999-1901.gz
|  |- 010015-99999-1901.gz
|  |- 010016-99999-1901.gz
|  |- 010017-99999-1901.gz
|  |- 010030-99999-1901.gz
|  |- 010040-99999-1901.gz
|  |- 010080-99999-1901.gz
|  |- 010100-99999-1901.gz
|- 1902
|- 1903
|- 1904
|- 1905
|- 1906
|- 1907
|- 1908
|- 1909
|- 1910
.
.
.
|- 2001
```

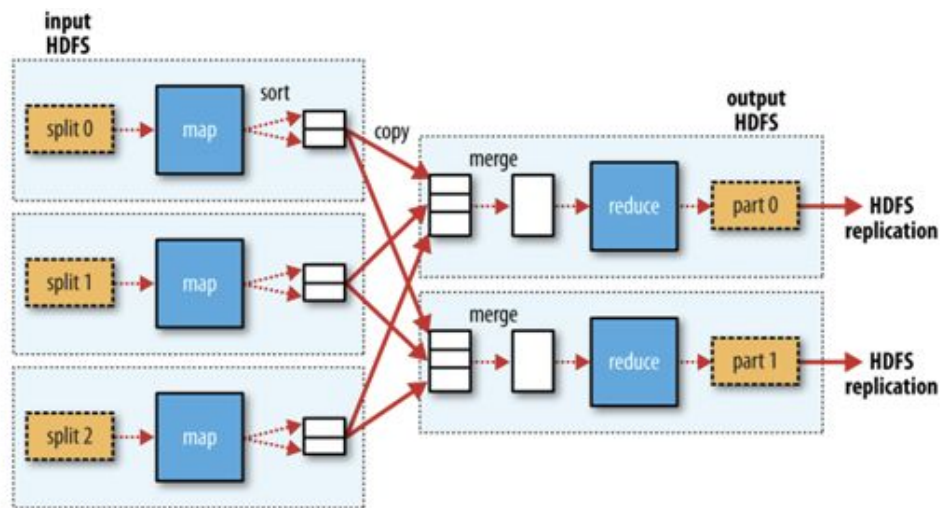
```
% ls raw/1990 | head
010010-99999-1990.gz
010014-99999-1990.gz
010015-99999-1990.gz
010016-99999-1990.gz
010017-99999-1990.gz
010030-99999-1990.gz
010040-99999-1990.gz
010080-99999-1990.gz
010100-99999-1990.gz
```

```
0067011990999991950051507004...9999999N9+00001+9999999999...
0043011990999991950051512004...9999999N9+00221+9999999999...
0043011990999991950051518004...9999999N9-00111+9999999999...
0043012650999991949032412004...0500001N9+01111+9999999999...
0043012650999991949032418004...0500001N9+00781+9999999999...
```

MapReduce

Moving computation is easier than moving data!

- The MapReduce paradigm contains two main steps
 - Map and Reduce
 - Work with data that are in key-value format
- Map:
 - Converts local raw data (splits) into key value pairs $\langle k1, v1 \rangle$
 - And sorts them by key
- Reduce:
 - Receives map results and combines them into the final answer $\langle k2, v2 \rangle$ based on the defined operation (e.g sum)



Moving data from mappers to reducers is called the 'sort/shuffle' phase

MapReduce: Calculating max temperature (cont.)

The Map function: extracts the year and the air temperature Combine same keys on mapper & send to reducers

```
0067011990999991950051507004...9999999N9+00001+9999999999...
0043011990999991950051512004...9999999N9+00221+9999999999...
0043011990999991950051518004...9999999N9-00111+9999999999...
0043012650999991949032412004...0500001N9+01111+9999999999...
0043012650999991949032418004...0500001N9+00781+9999999999...
```



Map

```
(1950, 0)
(1950, 22)
(1950, -11)
(1949, 111)
(1949, 78)
```

Sort/Shuffle



```
(1949, [111, 78])
(1950, [0, 22, -11])
```

The Reduce function: picks out the maximum reading of each year

```
(1949, [111, 78])
(1950, [0, 22, -11])
```

Reduce



```
(1949, 111)
(1950, 22)
```

REDUCES THE AMOUNT OF DATA THAT MOVES!

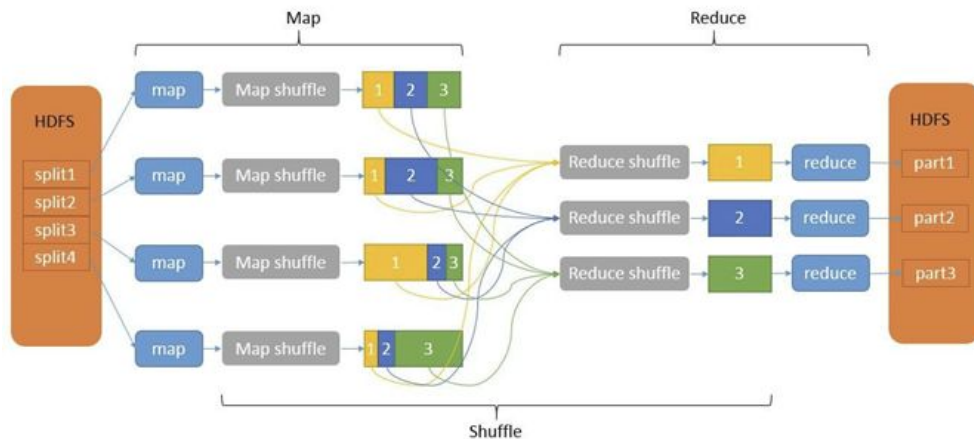
However:

- requires formulating the problem as operations on <key, value> items
- requires keys to be comparable (why?)

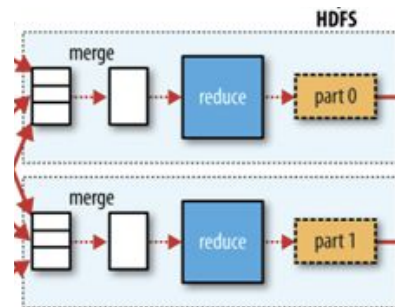
MapReduce: Shuffle / Sort

- The reduction operation ‘**reduces**’ all values associated with a key
[<date, temperature>] => <date : max_temeprature>
- We need to give the reducers all <key,value> pairs for their associated key
- This is the goal of the sort/shuffle part and the reason keys NEED to be comparable
- The nodes that execute the reduce are
 - The same that execute the mapp
 - The same that store the original data

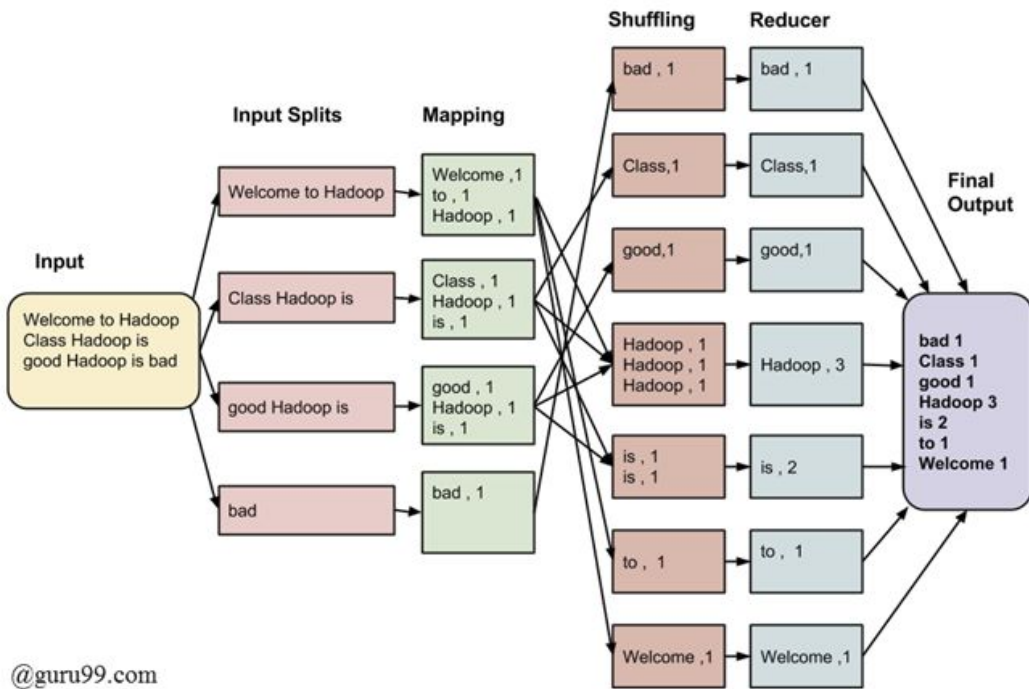
Doesn't that violate the “move computation not data” idea?
I guess a bit but... map HEAVILY compresses data



- **MapReduce produces as many result files (parts) as the reducers**



A more complete example: WordCount



MapReduce workflow:

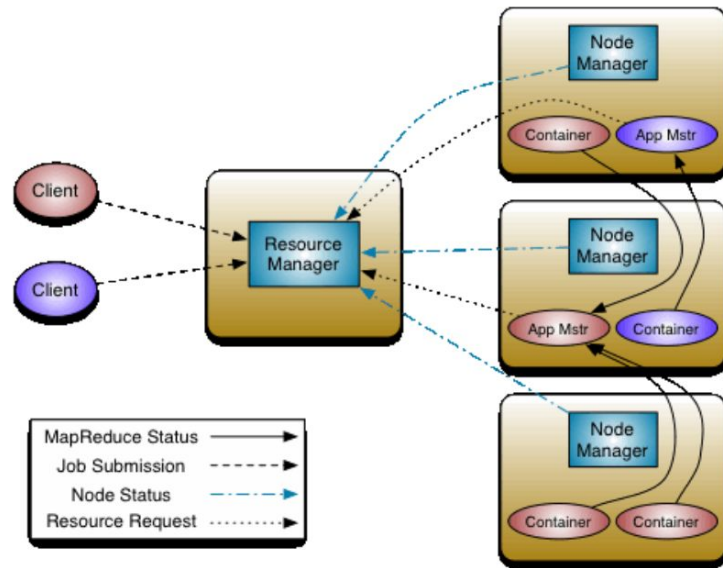
- **InputFormat:** splits blocks into input $\langle k, v \rangle$ for each mapper
- **Map:** maps input into $\langle k1, v1 \rangle$
- **Combiner (optional):** executes the reduction logic on each map output
- **Shuffle/Sort:** mappers send keys to reducers over the network
- **Reduce:** combine the received $\langle k1, v1 \rangle$ into final output $\langle k2, v2 \rangle$

How do we execute a MapReduce job?

YARN: Brief introduction

YARN is HADOOP's resource manager:

- Receives jobs request
- Determines the required resources (RAM, CPU cores etc..)
- Assigns the jobs to the hadoop nodes for execution



You can learn more about YARN in Apache's documentation:

<https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/YARN.html>

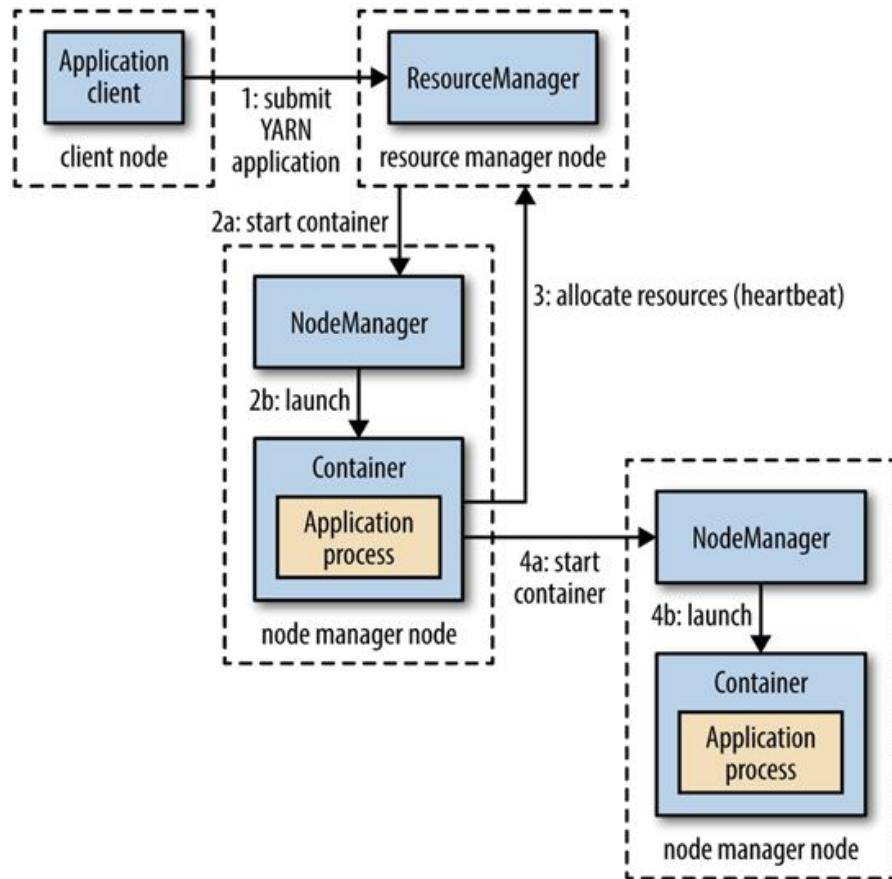
YARN: Components & Workflow

ResourceManager:

- One per cluster
- The central controller, allocate resources, submitting & managing the jobs

NodeManager:

- One per node in the cluster
- Monitors containers that execute the application-specific process with a constrained set of resources (CPU, memory, disk, network).
- Reports the resource usage to the ResourceManager



TASK: MapReduce on our docker cluster

When creating the datanodes notice that the local resources directory is mounted to the container!

Part 1: Execute a pre-compiled example that calculates pi digits

- Inspect the calculate_pi.sh script to understand how to submit a mapreduce job to YARN
- Connect to a datanode 'make connect'
- Ensure the script is executable 'sudo chmod +x <path/to/script>'
- Execute the script from the working dir of the container
 - ./resources/calculate_pi.sh <number_of_digits_to_compute> <output_path_in_HDFS>
- Checkout the job in: <http://localhost:8088/cluster>

Part 2: Execute a costume MapReduce job for counting words

- This is similar to the last one but here focus on understanding the code in WordCount.java
 - The following link show you how to write MapReduce jobs in Java
 - <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>
- Before running the 'word_count.sh' you need to upload your input data to HDFS (you can use 'upload.sh')
 - Read the script first to figure out the parameters you need to provide

NOTE:

MapReduce jobs DO NOT replace output directories. If you try to use an existing path you will get an error.